

AI_HW1

1、

1.1

增加了探索集，避免了重复性的搜索。

1.2

DFS：空间复杂度低，为 $O(bm)$ ，但是时间复杂度高，为 $O(b^m)$ ，且不能处理带有环的问题，不具备完备性和最优性。

UCS：空间复杂度有所降低，为 $O(b^{G^*/\epsilon})$ ，时间复杂度有所升高，为 $O(b^{G^*/\epsilon})$

1.3

约束多的变量剩余的可供选择少，如果后选这些变量的话，该过程很大程度会fail；选择约束最少的值是为了“与人行方便”，避免其他变量无路可走。

1.4

提高多样性，减小进入局部极值的可能性。不一定能找到最优解。

1.5

不会，因为 $\alpha - \beta$ 剪支后，该分支不会再被考虑，即使其值发生变化，也不会对决策产生影响。

2、

2.1

证明：

对于最优路径上位于边缘集中的点 n^* ，有 $h^*(S) = g(n^*) + h^*(n^*)$

则有：

$$f(n^*) = g(n^*) + h(n^*) \leq g(n^*) + h^*(n^*) + C_1$$

终点 T 一定位于最优路径上，当即将把 T 从边缘集中取出的时候，即代入 $n^* = T$ 有：

$$f(T) = g(T) + h(T) = g(T) \leq g(n^*) + h^*(n^*) + C_1 = h^*(S) + C_1$$

证毕。

2.2

证明：

对于最优路径上位于边缘集中的点 n^* ，有 $h^*(S) = g(n^*) + h^*(n^*)$

则有：

$$f(n^*) = g(n^*) + h(n^*) \leq g(n^*) + C_2 h^*(n^*)$$

终点 T 一定位于最优路径上，当即将把 T 从边缘集中取出的时候，即代入 $n^* = T$ 有：

$$f(T) = g(T) + h(T) = g(T) \leq g(n^*) + C_2 h^*(n^*) \leq C_2 g(n^*) + C_2 h^*(n^*) = C_2 h^*(S)$$

证毕。

3、

3.1

先手无必胜策略，后手可以保证不输。

3.2

在附件中

3.3

UCT	Random	$\alpha - \beta$	UCT
先手	胜率：81.9% 不输率：98.6%	胜率：0% 不输率：98%	胜率：3.6% 不输率：99.5%
后手	胜率：53.6% 不输率：94.2%	胜率：0% 不输率：9.9%	胜率：1.2% 不输率：95%

3.4

为了便于观察，分别对 C=0.1和 C=5.0的情况进行了两次对局，输出了各自的policy来进行对比。对战双方均选择 UCT。

C=0.1:

```
(base) PS F:\AI\hw1\code> python -m pit
Player 1:UCT Player  Player 2:UCT Player
0%| 0/1 [00:00<?, ?it/s]+
*****
[0.125 0.125 0.140625 0.125 0.078125 0.109375 0.109375 0.09375
0.09375 ]
-----
*****
[0.09375 0.15625 0. 0.25 0.09375 0.0625 0.09375 0.125 0.125 ]
-----
*****
[0.171875 0.09375 0. 0. 0.125 0.15625 0.15625 0.125
0.171875]
-----
*****
[0. 0.3030303 0. 0. 0.13636364 0.18181818
0.16666667 0.09090909 0.12121212]
-----
*****
[0. 0. 0. 0. 0.046875 0.796875 0.03125 0.046875
0.078125]
-----
*****
[0. 0. 0. 0. 0.26086957 0.
0.14492754 0.1884058 0.4057971 ]
-----
*****
[0. 0. 0. 0. 0.15116279 0.
0.03488372 0.81395349 0. ]
-----
*****
[0. 0. 0. 0. 0.5 0. 0.5 0. 0. ]
-----
*****
[0. 0. 0. 0. 0. 0. 1. 0. 0. ]
-----
P1 win: 0 (0) P2 win: 0 (0) Draw: 1 (1): 100%| 1/1 [00:00<00:00, 8.41it/s]
Player 1 (UCT Player) win: 0 (0.00%)
Player 2 (UCT Player) win: 0 (0.00%)
Draw: 1 (100.00%)
Player 1 not lose: 1 (100.00%)
Player 2 not lose: 1 (100.00%)
(base) PS F:\AI\hw1\code>
```

C=5.0:

```
(base) PS F:\AI\hw1\code> python -m pit
Player 1:UCT Player  Player 2:UCT Player
0%| 0/1 [00:00<?, ?it/s]+
*****
[0.125 0.109375 0.109375 0.109375 0.109375 0.109375 0.109375 0.109375
0.109375]
-----
*****
[0. 0.125 0.125 0.125 0.125 0.125 0.125 0.125 0.125]
-----
*****
[0. 0. 0.15625 0.140625 0.15625 0.140625 0.140625 0.125
0.140625]
-----
*****
[0. 0. 0. 0.140625 0.171875 0.171875 0.171875 0.171875
0.171875]
-----
*****
[0. 0. 0. 0.203125 0. 0.1875 0.1875 0.234375
0.1875 ]
-----
*****
[0. 0. 0. 0.26153846 0. 0.27692308
0.23076923 0. 0.23076923]
-----
*****
[0. 0. 0. 0.51515152 0. 0.
0.24242424 0. 0.24242424]
-----
*****
[0. 0. 0. 0. 0. 0.
0.72463768 0. 0.27536232]
-----
*****
[0. 0. 0. 0. 0. 0. 0. 1. ]
-----
P1 win: 0 (0) P2 win: 0 (0) Draw: 1 (1): 100%| 1/1 [00:00<00:00, 7.61it/s]
Player 1 (UCT Player) win: 0 (0.00%)
Player 2 (UCT Player) win: 0 (0.00%)
Draw: 1 (100.00%)
Player 1 not lose: 1 (100.00%)
Player 2 not lose: 1 (100.00%)
```

可以看出，C较大时比 C 较小时，policy分布更加均匀，这是因为在 UCT 公式中，C 所在的后半部分部分为探索部分，前半部分为利用部分。C 取值较小 MCTS 更倾向于利用当前已知的最优策略，而较少尝试新路径；C 取值较大 MCTS 更倾向于尝试新的未充分探索的动作，而不是立即选择当前最优的策略。

3.5

UCT 对 Random，先手胜率75%，不输率79%；后手胜率71%，不输率76%。所取参数为 n_rollout=7, n_search=64。

n_search（搜索次数）：

MCTS 在每一步执行的搜索次数。每次搜索会从根节点出发，经过 UCT 选择、扩展、仿真和回溯，最终决定动作的选择概率。

较大值： 提高策略质量，但计算更慢。

较小值： 计算更快，但策略质量下降。

n_rollout (仿真次数)：

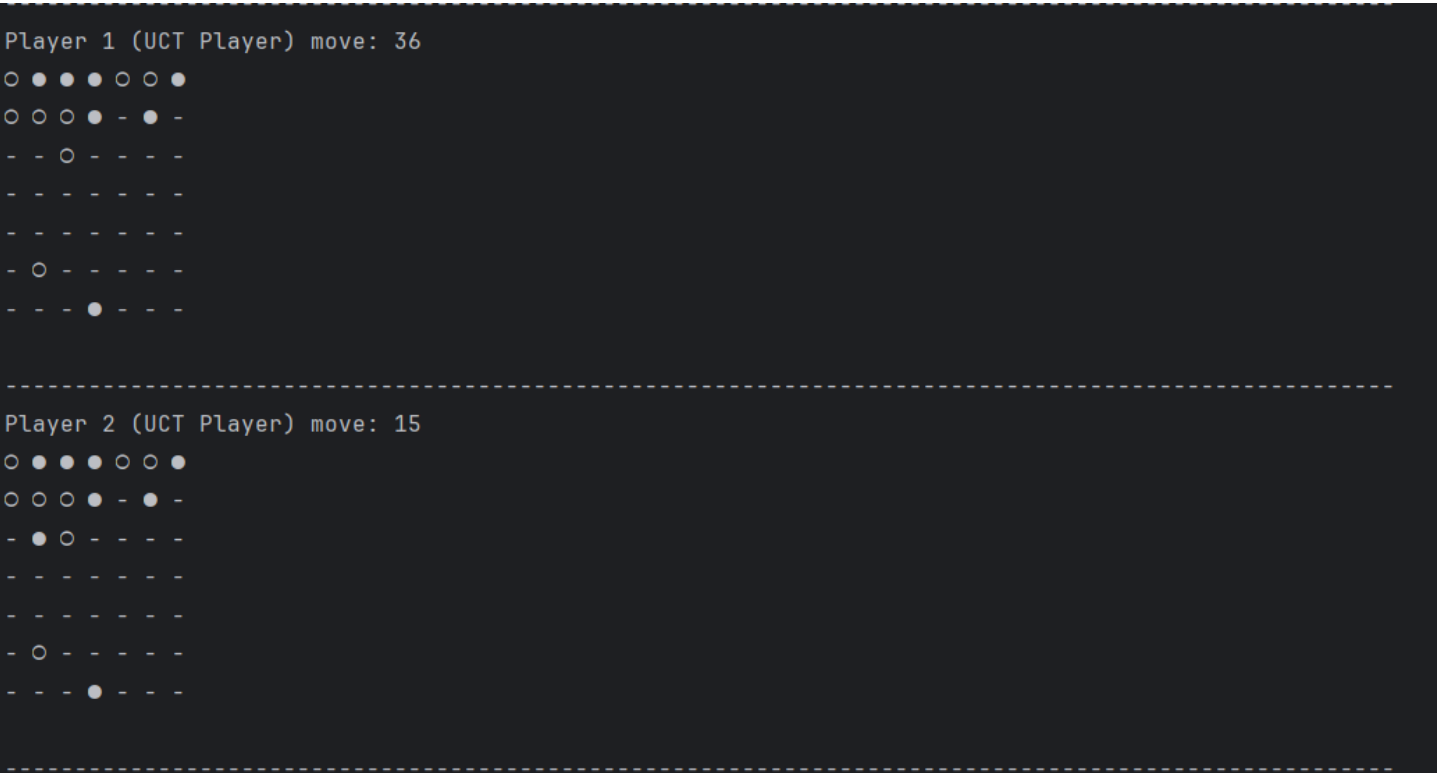
在叶节点执行的随机对局次数，用于估计局势。

较大值： 估值更准确，但计算更慢。

较小值： 估值噪声较大，影响策略稳定性。

3.6

使用 UCT 与 UCT 对弈。



从以上部分结果可以看出，并不是每一步都合理，例如，白子上面的情况下，选择在(2, 1)落子，可是如果在(1, 4)落子，就可以去掉黑方两个子。

3.7

(a) 在附件中。

(b) 在附件中。

(c) 我的实现是前十步正常，十步之后再使用启发式。当使用启发式后，MCTS 的效果肯定会下降，胜率和输率都会降低，但是速度会明显提高。对不同的对手分别进行100局对弈。

MCTS 对 Random:

```
(base) PS F:\AI\hw1\code> python -m pit
Player 1:UCT Player  Player 2:Random Player
P1 win: 69 (69) P2 win: 27 (27) Draw: 4 (4): 100%|
Player 1 (UCT Player) win: 69 (69.00%)
Player 2 (Random Player) win: 27 (27.00%)
Draw: 4 (4.00%)
Player 1 not lose: 73 (73.00%)
Player 2 not lose: 31 (31.00%)
Player 1:Random Player  Player 2:UCT Player
P1 win: 36 (36) P2 win: 54 (54) Draw: 10 (10): 100%|
Player 1 (Random Player) win: 36 (36.00%)
Player 2 (UCT Player) win: 54 (54.00%)
Draw: 10 (10.00%)
Player 1 not lose: 46 (46.00%)
Player 2 not lose: 64 (64.00%)
```

MCTS 对 $\alpha - \beta$:

```
(base) PS F:\AI\hw1\code> python -m pit
Player 1:UCT Player  Player 2:AlphaBeta Heuristic Player
P1 win: 0 (0) P2 win: 99 (99) Draw: 1 (1): 100%|
Player 1 (UCT Player) win: 0 (0.00%)
Player 2 (AlphaBeta Heuristic Player) win: 99 (99.00%)
Draw: 1 (1.00%)
Player 1 not lose: 1 (1.00%)
Player 2 not lose: 100 (100.00%)
Player 1:AlphaBeta Heuristic Player  Player 2:UCT Player
P1 win: 100 (100) P2 win: 0 (0) Draw: 0 (0): 100%|
Player 1 (AlphaBeta Heuristic Player) win: 100 (100.00%)
Player 2 (UCT Player) win: 0 (0.00%)
Draw: 0 (0.00%)
Player 1 not lose: 100 (100.00%)
Player 2 not lose: 0 (0.00%)
```

从上面可以看出，棋力强弱排序为：Random < HeuristicMCTS < HeuristicAlphaBeta。

4、

4.1

MCTS 通过统计采样进行搜索，可以重点探索有前景的分支，适用于高分支因子的博弈（如围棋）。不需要完整的状态评估函数，可以结合神经网络的估值进行强化学习。

Alpha-Beta 依赖一个 强大的静态评估函数，如果评估不准确，搜索的质量就会下降。适用于分支较少、搜索深度可控的情况（如国际象棋），但在围棋等复杂游戏中，状态评估函数难以构造，效果较

差。

此外，MCTS 可与深度学习更好结合。结合策略网络指导行动选择，避免遍历所有分支（Alpha-Beta 需要明确的启发式）。结合价值网络进行评估，而不是搜索到终局再计算胜负。

4.2

UCT 是传统的 MCTS 选择策略，使用 UCB1 公式来平衡探索（exploration）和开发（exploitation）：

$$Q(s, a) + C \sqrt{\frac{\log(N(s))}{N(s, a)}}$$

适用于一般 MCTS 算法，在搜索过程中通过 UCB1 选择最优探索-开发平衡的子节点。

适合无模型（model-free）环境，不依赖外部知识，只使用搜索统计信息进行决策。

AlphaZero 采用的 PUCT（Predictor + UCT），在 UCT 基础上结合了神经网络的策略先验来指导搜索。PUCT 选择公式如下：

$$Q(s, a) + C_{PUCT} \cdot P(s, a) \cdot \frac{\sqrt{N(s)}}{1 + N(s, a)}$$

利用神经网络的先验概率 $P(s, a)$ ，而不是均匀探索所有分支，使搜索更高效。

在探索时优先选择神经网络认为好的动作，减少随机探索，提高搜索质量。

适合有模型（model-based）的 AlphaZero 框架，结合深度学习不断优化策略。